

LAB-1: 机器启动

【注：本文件为助教建议前的原README内容，写得有点过于详细了，类似实验报告而不是README，后来改掉了】

lab1的核心目标是**机器启动**，具体来讲，我们主要完成了：

1. 通过BIOS→_entry→start→main**进入main函数**
2. 借助MMIO读写串口设备（UART）寄存器以完成**字符打印**，并借助stdarg.h的va_list类型实现**printf格式化输出**
3. 通过编写自旋锁函数并在printf中使用以实现双核的**有序输出**

我们的分工如下：

- **吴晨曦**：完成了 `start.c` 和 `print.c`，以及对应的实验文档。
- **丁熙妍**：完成了 `spinlock.c` 和 `main.c`，以及对应的实验文档。

1. 代码组织结构

```
ECNU-OSLAB-2025-TASK
├── LICENSE          开源协议
├── .vscode          配置了可视化调试环境
├── registers.xml    配置了可视化调试环境
├── Makefile         编译运行整个项目
├── common.mk        Makefile中一些工具链的定义
├── kernel.ld       定义了内核程序在链接时的布局
├── pictures         README使用的图片目录
├── README.md        实验指导书
├── src              源码
│   └── kernel      内核源码
│       ├── arch    RISC-V相关
│       │   ├── method.h
│       │   ├── mod.h
│       │   └── type.h
│       ├── boot    机器启动
│       │   ├── entry.S
│       │   └── start.c (本实验完成)
│       ├── lock    锁机制
│       │   ├── spinlock.c (本实验完成)
│       │   ├── method.h
│       │   ├── mod.h
│       │   └── type.h
│       ├── lib     常用库
│       │   ├── cpu.c
│       │   ├── print.c (本实验完成)
│       │   ├── uart.c
│       │   ├── method.h
│       │   ├── mod.h
│       │   └── type.h
│       └── main.c (本实验完成)
```

2. 进入main函数

依据`kernel.ld`中的`ENTRY(_entry)`和`. = 0x80000000`两行代码，QEMU启动时，CPU的PC会被设置为`0x80000000`，即`_entry`标签的地址。`_entry`标签被定义在`entry.S`汇编文件中，`entry.S`文件为函数栈顶指针设置了合适的值，并且调用了`start函数` `call start`。

然后程序进入`start.c`文件中的`start()`函数，在`start()`函数中，程序需要**进入main函数**并且从**M-mode进入S-mode**，为了从更高特权级的M-mode降级到S-mode，需要**利用异常返回机制手动构造一个异常返回环境**

- 首先通过`w_mstatus(status)`修改`m_status`寄存器，假装在发生“（假）异常”前的环境状态为S-mode
- 然后通过`mepc`设定返回原先环境后的下一条指令，即**设定为main函数**
- 最后使用`mret`回到之前的环境状态（即**我们假装的S-mode**）

总体`start.c`的文件结构如下：

```
start():
    w_satp(0);           // 关闭分页
    w_tp(hartid);        // 保存核心ID（只有m-mode可以访问ID）
    w_mstatus(MPP=S);    // 设置返回模式为 S-mode
    w_mepc(main);        // 设置返回地址为 main
    mret                 // 跳转到 S-mode 并执行 main()
    ↓
main()                  // 操作系统主逻辑开始！
```

然后我们就成功通过BIOS→`_entry`→`start()`→`main`进入到了main函数！

3. 串口与格式化输出

为了证明我们进入了main函数，我们需要让main函数打印点东西。

3.1 串口输出

为了将字符打印到终端，需要让**驱动程序控制串口设备（UART）**，控制串口设备也就是**读写串口设备的寄存器**。CPU无法直接访问这些物理寄存器，然而**MMIO（Memory-Mapped I/O）**技术能够把设备的寄存器“映射”到一段特定的内存地址空间上，于是驱动程序可以**像读写普通内存一样读写外部设备的寄存器**，从而实现了对外部串口设备的控制。

```
// type.h头文件中的相关宏定义
#define Reg(reg) ((volatile unsigned char *) (UART_BASE + reg))
#define WriteReg(reg, v) (*(Reg(reg)) = (v))
```

- `Reg`函数的作用是把“**寄存器编号**”转换成该寄存器在**内存中的实际地址（指针）**
- `WriteReg`的作用是将值 `v` **写入 UART（或其他外设）的某个寄存器 `reg`**

具体的**串口输入/输出函数**在`uart.c`中实现，使用了以上两个宏定义函数。

```
//uart.c中实现的串口输出接口
void uart_init(void); //uart 初始化
void uart_putc_sync(int c); // 单个字符输出
int uart_getc_sync(void); // 单个字符输入
void uart_intr(void); // 中断处理(键盘输入->屏幕输出)
```

在这些串口输出接口的基础上，加上对可变参数的解析可以实现下面的 `printf` 格式化输出

3.2 格式化输出

在UART接口的基础上，使用 `stdarg.h` 和 `va_list` 类型来逐个获得 `printf` 函数的 **可变参数**

`stdarg.h` 是编译器自带的库，其中的 `va_list` 类型有以下使用函数：

```
#include <stdarg.h>
va_list ap; // 参数指针
va_start(ap, fmt); // 指向第一个可变参数
va_arg(ap, type); // 获取下一个类型为 type 的参数
va_end(ap); // 清理（可选）
```

在编写 `printf` 函数时，首先声明一个 `va_list ap` 作为 **参数指针**，用于 **遍历可变参数**，再声明一个 `const char *p` 作为 **字符指针**，用于 **遍历格式字符串**，识别到 `%` 就从可变参数中取出对应值并打印。`printf` 函数的核心代码及注释如下：

```
for(p = fmt; *p; p++){
    //若不是格式化字符，直接输出
    if(*p != '%'){
        uart_putc_sync(*p);
        continue;
    }
    //若识别到格式化字符
    p++;
    switch(*p){
        case 'd':
            printint(va_arg(ap, int), 10, 1);
            break;
        case 'p':
            printint(va_arg(ap, uint32), 16, 0);
            break;
        case 'x':
            printptr(va_arg(ap, uint64));
            break;
        case 'c':
            c = va_arg(ap, int); // char会被提升为int
            uart_putc_sync(c);
            break;
        case 's':
            s = va_arg(ap, char*);
            if(s == 0)
                s = "(null)"; //若为空字符串
            while(*s != '\0'){
                uart_putc_sync(*s);
                s++;
            }
    }
}
```

```

        break;
    default:
        uart_putc_sync('%');
        uart_putc_sync(*p);
        break;
    }
}

```

然后在main中写出以下程序来验证 `printf` 函数的格式化输出情况：

```

printf("Hello!This is our OS Kernel!This Kernel is written by %d people:%s and
%s.\n \
Let's try and print %c,%p and %x.\n \
We made it!It's amazing!!\n", \
2, "dxy", "wcx", 'A', 0x12345678U, 0x1234567890abcdefULL);

```

运行结果如下图所示：

```

HHello!This is our OS Kernel!This Kernel is writtenello!This is our OS Kernel!This Kernel is written by 2 people:dxy and wcx.
Let's try and print A,1 by 2 people:dxy 2345678 and 0x1234567890abcdef.
We made it!It's amazing!!
and wcx.
Let's try and print A,12345678 and 0x1234567890abcdef.
We made it!It's amazing!!

```

可以看出，程序成功打印出了格式化内容，但是出现了混乱交错的现象，因此还需要添加一种同步机制来协调共享资源的有序使用。

4.自旋锁

4.1 自旋锁接口的实现

4.1.1 `spinlock_init`：锁的初始化

初始化自旋锁：

- 将锁的状态设为 0，表示未上锁；
- 将锁的名字设为 `name`；
- 将持有锁的cpu id 设为 -1，表示无cpu持有该锁。

```

// 自旋锁初始化
void spinlock_init(spinlock_t *lk, char *name)
{
    // 初始状态：未上锁（无cpu持有该锁）
    lk->locked = 0;
    lk->name = name;
    lk->cpu_id = -1;
}

```

4.1.2 `spinlock_holding`: 是否持有锁

辅助函数:

用于判断当前 CPU 是否正持有指定的锁。

```
// 是否持有自旋锁
bool spinlock_holding(spinlock_t *lk)
{
    // 检查当前的cpu是否持有该锁
    return lk->locked && lk->cpu == mycpu();
}
```

4.1.3 `spinlock_acquire`: 获取锁

获取指定锁，如果锁已被占用，则原地等待。

- 在获取锁之前使用 `push_off()` 关闭中断，防止死锁。
- 使用 `assert` 和 `spinlock_holding` 确保当前cpu没有持有该锁，否则无需再获取该锁。
- 使用原子操作 `__sync_lock_test_and_set(&lk->locked, 1)` 获取锁，它将 `lk->locked` 的值设置为 1，并返回设置前的旧值。
 - 如果旧值是 0，表示锁正空闲，则成功获取锁，循环结束。
 - 如果旧值是 1，表示锁已被占用，则继续在 `while` 循环中“自旋”等待，直到该锁空闲。
- 使用原子操作 `__sync_synchronize()` 作为内存屏障，确保临界区代码不会被乱序执行到锁的外面。
- 最后成功获取锁之后，更新持有该锁的cpu信息。

```
// 获取自旋锁
// 原地循环，直到能够获取锁
void spinlock_acquire(spinlock_t *lk)
{
    // 关中断
    push_off();

    // 确保当前cpu没有已经持有该锁
    assert(!spinlock_holding(lk), "spinlock_acquire: lock is already held by this
cpu\n");

    // 自旋等待，直到成功获取锁
    while (__sync_lock_test_and_set(&lk->locked, 1) != 0) // 原子操作：尝试获取锁
        ;

    // 防止指令的乱序执行导致锁机制失效
    __sync_synchronize();
    // 记录该锁的cpu信息

    lk->cpu = mycpu();
}
```

4.1.4 `spinlock_release`: 释放锁

释放指定的已持有的锁:

- 使用 `assert` 和 `spinlock_holding` 确保只有该锁的持有者才能释放锁。
- 清除持有该锁的cpu信息。
- 使用原子操作 `__sync_synchronize()` 作为内存屏障。
- 使用原子操作 `__sync_lock_release(&lk->locked)` 释放锁: 将 `lk->locked` 的值设置为 0, 让其他等待的cpu可以获取该锁。
- 使用 `pop_off()` 恢复之前的中断状态。

```
// 释放自旋锁
void spinlock_release(spinlock_t *lk)
{
    // 确保当前cpu持有锁
    assert(spinlock_holding(lk), "spinlock_release: lock is not held by this
cpu\n");

    // 清除该锁的cpu信息
    lk->cpu_id = -1;

    __sync_synchronize();

    // 释放锁
    __sync_lock_release(&lk->locked); // 原子操作

    // 开中断
    pop_off();
}
```

4.2 在printf函数中使用自旋锁

在printf函数中使用自旋锁以实现有序输出, 即需要在开始解析并输出格式字符串前先“上锁”, 然后在解析输出完后再“解锁”。

```
spinlock_acquire(&print_lk); // 上锁

for(p = fmt; *p; p++){ // 解析并输出格式字符串
    switch(*p){
        case 'b':
            ...
        case ...
            ...
    }
}

spinlock_release(&print_lk); // 解锁
```

4.2.1 双核的机器启动

使用完自旋锁后，可完成双核启动的main.c:

```
volatile static int started = 0;

int main()
{
    int cpuid = r_tp();
    if (cpuid == 0) {
        print_init();
        printf("CPU %d is booting!\n", cpuid);
        __sync_synchronize();
        started = 1;
    } else {
        while (started == 0);
        __sync_synchronize();
        printf("CPU %d is booting!\n", cpuid);
    }
    while (1);
}
```

结果如下:

```
qemu-system-riscv64 -machine virt -bios none -kernel target/kernel/kernel-qemu.elf -m 128M -smp
2 -nographic
CPU 0 is booting!
CPU 1 is booting!
```

成功完成了两个核的机器启动!

4.2.2 有序输出

使用完自旋锁后，针对3.2中同样的测试用例，打印结果如下:

```
qemu-system-riscv64 -machine virt -bios none -kernel target/kernel/kernel-qemu.elf
✓ Hello!This is our OS Kernel!This Kernel is written by 2 people:dxy and wx.
  Let's try and print A,12345678 and 0x1234567890abcdef.
  We made it!It's amazing!!
✓ Hello!This is our OS Kernel!This Kernel is written by 2 people:dxy and wx.
  Let's try and print A,12345678 and 0x1234567890abcdef.
  We made it!It's amazing!!
```

成功完成了两个核的有序输出!

4.3 并行加法

```
volatile static int started = 0;

volatile static int sum = 0;

int main()
{
    int cpuid = r_tp();
    if(cpuid == 0) {
        print_init();
        printf("cpu %d is booting!\n", cpuid);
        __sync_synchronize();
```

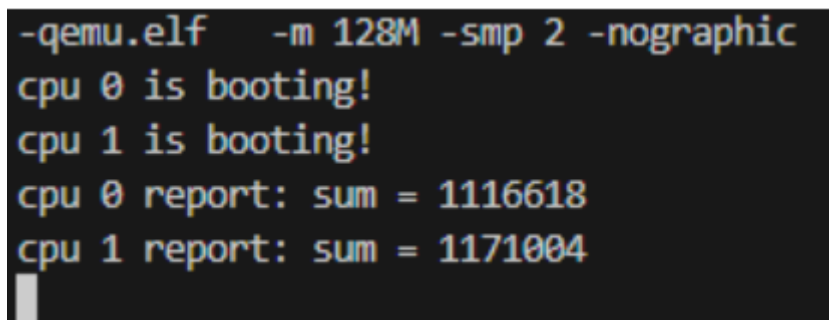
```

        started = 1;
        for(int i = 0; i < 1000000; i++)
            sum++;
        printf("cpu %d report: sum = %d\n", cpuid, sum);
    } else {
        while(started == 0);
        __sync_synchronize();
        printf("cpu %d is booting!\n", cpuid);
        for(int i = 0; i < 1000000; i++)
            sum++;
        printf("cpu %d report: sum = %d\n", cpuid, sum);
    }
    while (1);
}

```

在 `main.c` 中测试上述代码，很明显，我们的预期是后report的cpu应该告诉我们 `sum = 2000000`

但是实际结果是这样的：



```

-qemu.elf -m 128M -smp 2 -nographic
cpu 0 is booting!
cpu 1 is booting!
cpu 0 report: sum = 1116618
cpu 1 report: sum = 1171004

```

这是并发编程中的一个经典场景 -- **竞态条件**：

在未加锁的代码中，`sum++` 这个**非原子操作**在底层会这样进行：

1. 读取 `sum` 的值到 CPU 寄存器。
2. 在寄存器中将值+1。
3. 将新值写回内存中的 `sum`。

因此当两个cpu并发执行时，可能会发生：

1. cpu 0 读取 `sum` (此时值为 `x`)。
2. cpu 1 也读取 `sum` (值仍为 `x`)。
3. cpu 0 计算得 `x + 1`，并将 `x + 1` 写回 `sum`。
4. cpu 1 计算得 `x + 1`，并将 `x + 1` 写回 `sum`。

此时两个cpu的执行了加法，但结果是 `sum` 只+1，而不是我们希望的+2。

为了解决这个问题，我们需要**利用锁**来保证 `sum++` “读-改-写”这三个步骤的**原子性**：

- 在 `sum++` 前先**获取**一个锁
- 在 `sum++` 后再**释放**掉这个锁

修改后的 `main.c` 如下：

```

volatile static int started = 0;
volatile static int sum = 0;

spinlock_t sum_lock; // 为sum定义一个锁

```

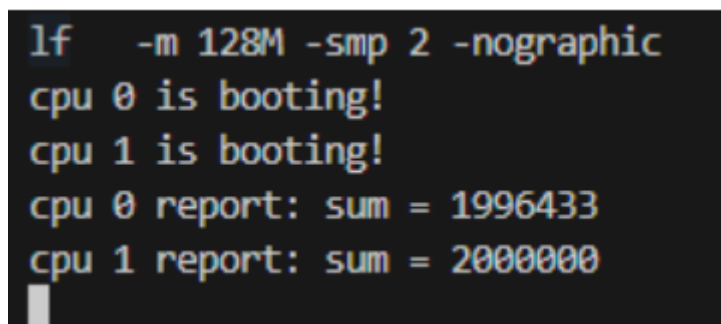


```

int main()
{
    int cpuid = r_tp();
    if(cpuid == 0) {
        print_init();
        spinlock_init(&sum_lock, "sum_lock"); // 初始化锁
        printf("cpu %d is booting!\n", cpuid);
        __sync_synchronize();
        started = 1;
        for(int i = 0; i < 1000000; i++)
        {
            spinlock_acquire(&sum_lock); // 在sum++前获取锁
            sum++;
            spinlock_release(&sum_lock); // 在sum++后释放锁
        }
        printf("cpu %d report: sum = %d\n", cpuid, sum);
    } else {
        while(started == 0);
        __sync_synchronize();
        printf("cpu %d is booting!\n", cpuid);
        for(int i = 0; i < 1000000; i++)
        {
            spinlock_acquire(&sum_lock); // 在sum++前获取锁
            sum++;
            spinlock_release(&sum_lock); // 在sum++后释放锁
        }
        printf("cpu %d report: sum = %d\n", cpuid, sum);
    }
    while (1);
}

```

修改后的输出如下：



```

lf -m 128M -smp 2 -nographic
cpu 0 is booting!
cpu 1 is booting!
cpu 0 report: sum = 1996433
cpu 1 report: sum = 2000000

```

成功完成了两个核的并行加法！

但这不是唯一解法，刚才是在**for循环内**加锁和解锁，而我们还可以在**for循环外**加锁和解锁：

```

spinlock_acquire(&sum_lock); // 在循环外获取锁
for(int i = 0; i < 1000000; i++)
    sum++;
spinlock_release(&sum_lock); // 在循环外释放锁

```

输出如下：

```
lf -m 128M -smp 2 -nographic
cpu 0 is booting!
cpu 1 is booting!
cpu 0 report: sum = 1000000
cpu 1 report: sum = 2000000
```

也成功完成了两个核的并行加法！

但可以看出，**上锁和解锁的位置不同**会极大地影响程序的行为和性能，这被称为**锁的粒度**。

1. 细粒度锁：在 for 循环内部加锁和解锁

```
for(int i = 0; i < 1000000; i++) {
    spinlock_acquire(&sum_lock); // 循环内加锁
    sum++;
    spinlock_release(&sum_lock); // 循环内解锁
}
```

- 行为：
两个cpu是**同时、交替**地对 `sum` 进行累加。每个cpu在每次循环中的会尝试获取锁，获取到的会执行 `sum++`，执行完后会立即释放锁。
- 并行度：
锁的持有时间极短，只保护了 `sum++` 这个最小的临界区。循环变量 `i` 的增减和判断等操作仍然是并行执行的，因此**并行度高**。
- 性能开销：
每个cpu都要执行100万次加锁和解锁，**总的锁操作开销巨大**。

2. 粗粒度锁：在 for 循环外部加锁和解锁

```
spinlock_acquire(&sum_lock); // 循环外加锁
for(int i = 0; i < 1000000; i++) {
    sum++;
}
spinlock_release(&sum_lock); // 循环外解锁
```

- 行为：
一个cpu（如cpu 0）先获取了锁，然后**独自完成了全部100万次累加**，此时`sum`变为1000000，再释放锁。在这个过程中，另一个cpu（cpu 1）**只能不停地“自旋”等待**。当cpu 0释放后，cpu 1才能获取锁，继续从1000000开始累加，最终得到2000000。
- 并行度：
实际上是cpu 0先做，做完之后cpu 1再做，因此本质上是**串行**，而不是并行。
- 性能开销：
每个cpu只需要执行一次加锁和解锁，**锁操作的开销极小**。

因此，选择锁的粒度是在“**锁操作开销**”和“**并行度**”之间的一种平衡：

锁的粒度越细，临界区越小，允许多个核心同时执行的代码就越多，并行度越高；但加锁/解锁更频繁，导致锁操作开销更大。

4.4 补充测试: 锁的嵌套调用

在并行加法的循环中加入 `printf` 调用, 使得每个cpu在持有 `sum_lock` 的同时, 还会去竞争 `print_lk`, 检验多重锁环境下我们写的代码是否能正常工作。

在 `main.c` 中测试以下代码:

```
volatile static int started = 0;
volatile static int sum = 0;
spinlock_t sum_lock;

int main()
{
    int cpuid = r_tp();
    if(cpuid == 0) {
        print_init();
        spinlock_init(&sum_lock, "sum_lock");
        printf("CPU %d is booting!\n", cpuid);
        __sync_synchronize();
        started = 1;

        for(int i = 0; i < 20; i++)
        {
            spinlock_acquire(&sum_lock);
            sum++;
            // 在持有 sum_lock 的同时, 调用 printf (会尝试获取 print_lk)
            printf("CPU %d: sum is now %d, loop index is %d\n", cpuid, sum, i);
            spinlock_release(&sum_lock);
        }
        printf("CPU %d finished. Final sum: %d\n", cpuid, sum);
    } else {
        while(started == 0);
        __sync_synchronize();
        printf("CPU %d is booting!\n", cpuid);

        for(int i = 0; i < 20; i++)
        {
            spinlock_acquire(&sum_lock);
            sum++;
            // 在持有 sum_lock 的同时, 调用 printf
            printf("CPU %d: sum is now %d, loop index is %d\n", cpuid, sum, i);
            spinlock_release(&sum_lock);
        }
        printf("CPU %d finished. Final sum: %d\n", cpuid, sum);
    }
    while (1);
}
```

输出如下:

```
CPU 0 is booting!
CPU 0: sum is now 1, loop index is 0
CPU 1 is booting!
CPU 0: sum is now 2, loop index is 1
```

```
CPU 0: sum is now 3, loop index is 2
CPU 0: sum is now 4, loop index is 3
CPU 1: sum is now 5, loop index is 0
CPU 0: sum is now 6, loop index is 4
CPU 1: sum is now 7, loop index is 1
CPU 0: sum is now 8, loop index is 5
CPU 1: sum is now 9, loop index is 2
CPU 0: sum is now 10, loop index is 6
CPU 1: sum is now 11, loop index is 3
CPU 0: sum is now 12, loop index is 7
CPU 1: sum is now 13, loop index is 4
CPU 0: sum is now 14, loop index is 8
CPU 1: sum is now 15, loop index is 5
CPU 0: sum is now 16, loop index is 9
CPU 1: sum is now 17, loop index is 6
CPU 0: sum is now 18, loop index is 10
CPU 0: sum is now 19, loop index is 11
CPU 1: sum is now 20, loop index is 7
CPU 0: sum is now 21, loop index is 12
CPU 0: sum is now 22, loop index is 13
CPU 1: sum is now 23, loop index is 8
CPU 0: sum is now 24, loop index is 14
CPU 1: sum is now 25, loop index is 9
CPU 0: sum is now 26, loop index is 15
CPU 1: sum is now 27, loop index is 10
CPU 0: sum is now 28, loop index is 16
CPU 1: sum is now 29, loop index is 11
CPU 0: sum is now 30, loop index is 17
CPU 1: sum is now 31, loop index is 12
CPU 0: sum is now 32, loop index is 18
CPU 1: sum is now 33, loop index is 13
CPU 0: sum is now 34, loop index is 19
CPU 1: sum is now 35, loop index is 14
CPU 0 finished. Final sum: 35
CPU 1: sum is now 36, loop index is 15
CPU 1: sum is now 37, loop index is 16
CPU 1: sum is now 38, loop index is 17
CPU 1: sum is now 39, loop index is 18
CPU 1: sum is now 40, loop index is 19
CPU 1 finished. Final sum: 40
```

成功完成了锁的嵌套调用！